

確保跨雲端代理式企業部署作業安全無虞

來源：<https://codelabs.developers.google.com/next26/aiinfra-learning-pod/screen1-securing-cross-cloud-agentic?hl=zh-tw>

步驟數：12

目錄

1. 簡介
2. 事前準備
3. 複製 GitHub 存放區
4. 使用 Terraform 部署基礎架構
5. 使用 Skaffold 部署範例工作負載
6. 透過 Model Armor 套用 AI 防護機制
7. 設定 Gemini Enterprise
8. 在 Gemini Enterprise 中部署及新增代理
9. 測試代理
10. 選用：使用 Apigee 將 REST API 轉碼為 MCP
11. 清除所用資源
12. 恭喜

1. 簡介

在本程式碼實驗室中，您將使用 Agent Development Kit (ADK)、Agent Engine 和 Google Kubernetes Engine，安全地部署單一代理 / 多項工具。您將瞭解 Gemini Enterprise 中由使用者啟動的 AI 代理程式，如何安全地瀏覽 Google Cloud，並透過 GKE Gateway 和 Service Extensions，從 MCP 工具回覆中途編輯機密資料。

課程內容

- 使用 [OpenTelemetry](#) 檢測，將 [ADK](#) 房貸助理代理部署至 [Agent Engine](#)
- 將後端 MCP 伺服器部署至內部 Gateway 後方的 [Google Kubernetes Engine \(GKE\)](#)
- 使用 [PSC 介面](#)和 [DNS 對等互連](#)，將 Agent Engine 連線至專案虛擬私有雲
- 使用 [Apigee](#) MCP 探索 Proxy，將 REST API 公開為 MCP 工具
- 設定 [GKE 閘道](#)，透過 [Service Extensions](#) 控管代理程式輸出內容，並進行 [DLP](#) 遮蓋和 MCP 授權
- 透過 [Model Armor](#) 套用 AI 防護機制，篩選提示詞和回覆

需求條件

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案
- 熟悉 Terraform、Kubernetes 和 Python 的基本知識

本程式碼實驗室適用於想在企業環境中部署及保護代理工作流程的開發人員和資安專業人員。

步驟 2 · 約 3 分鐘

2. 事前準備

建立 Google Cloud 專案並啟用必要 API。

1. 在 Google Cloud 控制台的專案選取器頁面中，選取或建立 [Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何檢查 [專案是否已啟用計費功能](#)。

必要的 IAM 角色

本程式碼研究室假設您已具備 Google Cloud 雲端專案的「專案擁有者」角色。

啟用 API

1. 在 Google Cloud 控制台中，按一下「啟用 Cloud Shell」：如果您從未使用過 Cloud Shell，系統會顯示窗格，讓您選擇是否要在受信任的環境中啟動 Cloud Shell，以及是否要啟用加速功能。如果系統要求您授權 Cloud Shell，請點選「授權」。
2. 在 Cloud Shell 中，啟用所有必要的 API：

□□□

```
gcloud services enable \  
compute.googleapis.com \  
container.googleapis.com \  
dns.googleapis.com \  
certificatemanager.googleapis.com \  
dlp.googleapis.com \  
aiplatform.googleapis.com \  
discoveryengine.googleapis.com \  
apigee.googleapis.com
```

安裝依附元件

在 Cloud Shell 中，確認您已安裝必要工具。Terraform、`kubectl` 和 Go 通常會預先安裝。您需要安裝 `uv` (Python 套件管理工具) 和 Skaffold：

```
# Install uv  
curl -LsSf https://astral.sh/uv/install.sh | sh  
  
# Install Skaffold  
curl -Lo skaffold https://storage.googleapis.com/skaffold/releases/latest/skaffold-linux-  
amd64 && \  
sudo install skaffold /usr/local/bin/
```

設定環境變數

設定本程式碼研究室全程使用的下列環境變數：

```
export PROJECT_ID=$(gcloud config get project)  
export REGION=us-central1  
export LOCATION=${REGION}
```

建立公開 DNS 區域

本程式碼研究室需要專案中預先存在的公開 DNS 區域，才能套用 Terraform 設定。這個區域用於名稱伺服器委派，以便設定自動建立 [Certificate Manager](#) 的必要記錄。

在 Cloud Shell 中執行下列指令，建立儲存區：

```
gcloud dns managed-zones create "inference-gateway-zone" \
  --dns-name="gateway.example.com." \
  --description="Public zone for Inference Gateway" \
  --visibility="public" \
  --project="${PROJECT_ID}"
```

將 `gateway.example.com.` 替換成您控管的網域或子網域。您必須在網域註冊商處設定名稱伺服器委派，指向這個區域的 Cloud DNS 名稱伺服器。

步驟 3 · 約 2 分鐘

3. 複製 GitHub 存放區

1. 在本機電腦的終端機中，複製 `cloud-networking-solutions` 存放區：

```
git clone https://github.com/googleCloudPlatform/cloud-networking-solutions.git
```

2. 前往下載的存放區目錄：

```
cd cloud-networking-solutions/demos/service-extensions-gke-gateway
```

4. 使用 Terraform 部署基礎架構

您將使用 Terraform 佈建基礎網路、GKE 叢集和必要的 ID 設定。

1. 前往複製存放區中的 `terraform` 目錄：

```
cd terraform
```

2. 設定 Terraform 後端。建立 `backend.conf` 檔案，用於部分後端設定。將 替換成全域不重複的值區名稱。

```
cat <<EOF > backend.conf
bucket = "<YOUR_TERRAFORM_STATE_BUCKET>"
prefix = "terraform"
EOF
```

如果沒有狀態值區，可以執行 `gcloud storage buckets create gs://<YOUR_TERRAFORM_STATE_BUCKET>` 建立。

3. 複製範例變數檔案，並以專案值更新：

```
cp example.tfvars terraform.tfvars
```

4. 編輯 `terraform.tfvars` 並替換預留位置值。請替換下列項目：

- **project_id**：您的 Google Cloud 專案 ID。
- **organization_id**：您的 GCP 機構數字 ID。
- **dns_zone_domain**：您控管的網域 (例如 `demo.example.com.`)，結尾必須加上半形句號。

確認已啟用下列功能旗標 (範例檔案中已預先設定)：

- `enable_certificate_manager = true`
- `enable_model_armor = true`
- `enable_agent_engine = true`
- `enable_psc_interface = true`

5. 初始化並套用 Terraform 設定：

```
terraform init -backend-config=backend.conf
terraform plan -out=tfplan
terraform apply "tfplan"
```

5. 使用 Skaffold 部署範例工作負載

注意：為確保體驗流暢且可重現，本實驗室中的所有 MCP 伺服器都會部署在 GKE 上。在實際的企業情境中，這些通常會分散在各種雲端和地端部署環境中。

接著，將 MCP 伺服器和外部處理服務部署至 GKE 叢集。

1. 連線至 Terraform 建立的 GKE 叢集：

```
gcloud container clusters get-credentials gateway-cluster \
  --region=${REGION} \
  --project=${PROJECT_ID}
```

2. 返回專案根層級，然後設定 Kubernetes 資訊清單範本。複製範例設定，然後設定專案 ID 和基本網域：設定 `BASE_DOMAIN` 以符合您的環境。`BASE_DOMAIN` 應與 `dns_zone_domain` Terraform 變數相符 (不含尾端半形句點)。

```
export BASE_DOMAIN=example.com
```

3. 從範本產生 Kubernetes 資訊清單和 `skaffold.yaml`：

```
bash k8s/generate.sh
envsubst '${PROJECT_ID}' < skaffold.yaml.tpl > skaffold.yaml
```

4. 部署所有後端服務：

```
skaffold run -m legacy-dms,income-verification-api,corporate-email,dlp-ext-proc
```

5. 部署 Gateway 和 HTTPRoute 設定：

```
kubectl apply -k k8s/gateway-internal/
```


6. 透過 Model Armor 套用 AI 防護機制

您可以將內容安全決策 (例如移除有害提示或防止資料外洩) 委派給 [Model Armor](#)。

必要條件：授予 IAM 角色

您必須將必要角色授予 GKE Gateway 服務帳戶。服務帳戶的格式為 `service-GATEWAY_PROJECT_NUMBER@gcp-sa-dep.iam.gserviceaccount.com`。

執行下列指令，授予必要權限：

```
export GATEWAY_PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")

# Grant roles in the Gateway project
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member=serviceAccount:service-${GATEWAY_PROJECT_NUMBER}@gcp-sa-dep.iam.gserviceaccount.com \
  --role=roles/modelarmor.calloutUser

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member=serviceAccount:service-${GATEWAY_PROJECT_NUMBER}@gcp-sa-dep.iam.gserviceaccount.com \
  --role=roles/serviceusage.serviceUsageConsumer

# Grant role in the project containing Model Armor templates
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member=serviceAccount:service-${GATEWAY_PROJECT_NUMBER}@gcp-sa-dep.iam.gserviceaccount.com \
  --role=roles/modelarmor.user
```

建立 Model Armor 授權擴充功能

定義指向您所在區域 [Model Armor](#) 服務的擴充功能。將這項設定儲存為 `ma-content-authz-extension.yaml`。

匯出 Terraform 建立的 Model Armor 範本 ID。

```
export MA_TEMPLATE_ID=$(cd terraform && terraform output -raw model_armor_template_id)
```

```

cat >ma-content-authz-extension.yaml <<EOF
name: ma-ext
service: modelarmor.$LOCATION.rep.googleapis.com
metadata:
  model_armor_settings: '[
    {
      "response_template_id":
"projects/${PROJECT_ID}/locations/$LOCATION/templates/${MA_TEMPLATE_ID}",
      "request_template_id":
"projects/${PROJECT_ID}/locations/$LOCATION/templates/${MA_TEMPLATE_ID}"
    }
  ]'
failOpen: true
EOF

gcloud beta service-extensions authz-extensions import ma-ext \
  --source=ma-content-authz-extension.yaml \
  --location=$LOCATION

```

建立 Model Armor 授權政策

建立政策，將 Model Armor 擴充功能與 Agent Gateway 建立關聯。將這項設定儲存為 `ma-content-authz-policy.yaml`。

```

cat >ma-content-authz-policy.yaml <<EOF
name: ma-content-authz-policy
target:
  resources:
    - "projects/$PROJECT_ID/locations/$LOCATION/gateways/mortgage-gateway"
policyProfile: CONTENT_AUTHZ
action: CUSTOM
customProvider:
  authzExtension:
    resources:
      - "projects/$PROJECT_ID/locations/$LOCATION/authzExtensions/ma-ext"
EOF

gcloud network-security authz-policies import ma-content-authz-policy \
  --source=ma-content-authz-policy.yaml \
  --location=$LOCATION

```

步驟 7 · 約 5 分鐘

7. 設定 Gemini Enterprise

啟用可觀測性

抵押貸款代理程式會部署 OpenTelemetry 檢測功能，並預設啟用下列遙測環境變數：

- `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true`
- `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=true`
- `OTEL_TRACES_SAMPLER=parentbased_traceidratio`

若要瞭解如何在 Gemini Enterprise 中設定追蹤記錄和時距，請參閱「[管理可觀測性設定](#)」。

在 Gemini Enterprise 中啟用 Model Armor

對 Gemini Enterprise 助理套用 [Model Armor](#) 篩選功能，篩選使用者提示詞和模型回覆。全球 Gemini Enterprise 應用程式需要在 `us` 多地區使用 Model Armor 範本，因此 Terraform 會為此部署個別範本。

從 Terraform 輸出內容擷取範本名稱：

```
cd terraform
export GE_MA_TEMPLATE_NAME=$(terraform output -raw
model_armor_gemini_enterprise_template_name)
```

擷取 Gemini Enterprise 執行個體的應用程式 ID：

1. 前往 Google Cloud 控制台的「Gemini Enterprise」頁面。
2. 按一下導覽選單中的「應用程式」。
3. 複製 Gemini Enterprise 執行個體的 ID

將 ID 匯出為環境變數。

```
export APP_ID=<PASTE_APP_ID>
```

修補助理，啟用 Model Armor：

```
curl -X PATCH \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H "Content-Type: application/json" \
  -H "X-Goog-User-Project: ${PROJECT_ID}" \
  "https://global-
discoveryengine.googleapis.com/v1/projects/${PROJECT_ID}/locations/global/collections/default
_collection/engines/${APP_ID}/assistants/default_assistant?update_mask=customerPolicy" \
  -d '{
    "customerPolicy": {
      "modelArmorConfig": {
        "userPromptTemplate": "'"$GE_MA_TEMPLATE_NAME"'",
        "responseTemplate": "'"$GE_MA_TEMPLATE_NAME"'",
        "failureMode": "FAIL_OPEN"
      }
    }
  }'
```

將 `failureMode` 設為 `FAIL_OPEN`，允許在 Model Armor 無法使用時傳送要求；設為 `FAIL_CLOSED` 則會封鎖要求。

8. 在 Gemini Enterprise 中部署及新增代理

取得授權詳細資料

請按照下列步驟取得授權詳細資料。

1. 在 Google Cloud 控制台的「API 和服務」頁面中，前往「憑證」頁面。
2. 前往「憑證」。
3. 按一下「建立憑證」，然後選取「OAuth 用戶端 ID」。
4. 在「應用程式類型」中，選取「網頁應用程式」。
5. 在「已授權的重新導向 URI」部分，新增下列 URI：
 - <https://vertexaisearch.cloud.google.com/oauth-redirect>
 - <https://vertexaisearch.cloud.google.com/static/oauth/oauth.html>
6. 按一下「建立」。
7. 匯出用戶端 ID 和用戶端密鑰以進行部署。

```
export OAUTH_CLIENT_ID=<Client ID>
export OAUTH_CLIENT_SECRET=<Client Secret>
```

部署抵押貸款代理程式

`src/mortgage-agent/deploy_agent.py` 指令碼會將 ADK 代理部署至 Agent Engine，並視需要註冊至 Gemini Enterprise。如要瞭解 Gemini Enterprise 註冊流程的背景資訊，請參閱「[註冊及管理 A2A 代理](#)」。

從 Terraform 部署作業匯出變數。

```
export VPC_NAME=$(cd terraform && terraform output -raw vpc_name)
export PSC_ATTACHMENT=$(cd terraform && terraform output -raw
psc_interface_network_attachment_id)
export DNS_PEERING_DOMAIN=$(cd terraform && terraform output -raw
psc_interface_dns_peering_domain)
```

安裝依附元件並部署：

```
cd src/mortgage-agent
uv sync
```

將代理部署至 Vertex Agent Engine，並註冊至 Gemini Enterprise：

```
uv run python deploy_agent.py \
  --project=${PROJECT_ID} \
  --dms-mcp-url=https://dms.${DNS_PEERING_DOMAIN%}.mcp \
  --income-verification-url=https://income-verification.${DNS_PEERING_DOMAIN%}. \
  --email-mcp-url=https://email.${DNS_PEERING_DOMAIN%}.mcp \
  --network-attachment=projects/${PROJECT_ID}/regions/${REGION}/networkAttachments/${PSC_ATTACHMENT} \
  --dns-peering-domain=${DNS_PEERING_DOMAIN} \
  --dns-peering-target-project=${PROJECT_ID} \
  --dns-peering-target-network=${VPC_NAME} \
  --enable-agent-identity \
  --ge-deploy \
  --app-id=${APP_ID} \
  --oauth-client-id=${OAUTH_CLIENT_ID} \
  --agent-name=mortgage-agent
```

旗標參考

檢舉	預設	說明
<code>--project</code>	<code>\$PROJECT_ID</code>	GCP 專案 ID
<code>--dms-mcp-url</code>	(必填)	DMS MCP 伺服器 URL
<code>--income-verification-url</code>	(必填)	Income Verification API 基本網址；系統會自動附加 <code>/mcp</code>
<code>--email-mcp-url</code>	(必填)	電子郵件 MCP 伺服器網址
<code>--region</code>	<code>\$REGION</code>	GCP 區域
<code>--staging-bucket</code>	<code>gs://PROJECT-staging</code>	用於暫存的 GCS bucket
<code>--display-name</code>	Mortgage Assistant Agent	已部署代理的顯示名稱
<code>--update</code>	(選填)	就地更新現有代理程式 (傳遞完整資源名稱)
<code>--network-attachment</code>	(選填)	PSC 介面的網路連結 (完整路徑或名稱)
<code>--dns-peering-domain</code>	(選填)	PSC-I DNS 對等互連的 DNS 網域 (必須以半形句號結尾)

<code>--dns-peering-target-project</code>	(選填)	代管 DNS 對接目標虛擬私有雲網路的專案
<code>--dns-peering-target-network</code>	(選填)	用於 DNS 對等互連的虛擬私有雲網路名稱
<code>--enable-agent-identity</code>	<code>false</code>	啟用各代理程式的最低權限憑證
<code>--ge-deploy</code>	<code>false</code>	部署後在 Gemini Enterprise 中註冊代理
<code>--app-id</code>	(選填)	Gemini Enterprise 引擎 ID (使用 <code>--ge-deploy</code> 時為必填)
<code>--oauth-client-id</code>	<code>\$OAUTH_CLIENT_ID</code>	OAuth2 用戶端 ID (使用 <code>--ge-deploy</code> 時為必填)
<code>--oauth-client-secret</code>	<code>\$OAUTH_CLIENT_SECRET</code>	OAuth2 用戶端密鑰 (使用 <code>--ge-deploy</code> 時為必填)
<code>--model</code>	<code>gemini-3.1-flash-lite-preview</code>	代理程式的 Gemini 模型名稱
<code>--agent-name</code>	<code>mortgage-agent</code>	Gemini Enterprise 授權和代理程式名稱
<code>--ge-deploy-only</code>	(選填)	在 Gemini Enterprise 中註冊現有的推論引擎，不必重新部署 (傳遞完整資源名稱)

新增具備權限的使用者

如要使用 [Google Cloud 控制台](#) 將具備權限的使用者新增至 ADK 代理程式，請參閱「新增或修改使用者及其權限」。



9. 測試代理

您已部署及設定代理程式、GKE Gateway 和所有後端服務，現在請測試端對端流程，確認安全政策是否正常運作。您將在 Gemini Enterprise 介面中與「mortgage-agent」互動。

存取代理程式

1. 前往 Google Cloud 控制台的「Gemini Enterprise」頁面。
2. 選取先前設定的 Gemini Enterprise 應用程式，其中已註冊「mortgage-agent」。
3. 在「總覽」分頁中，瀏覽「您的 Gemini Enterprise 網頁應用程式已準備就緒」部分顯示的網址。
4. 從左側選單 (稱為「代理庫」) 選取「代理程式」分頁標籤
5. 你現在應該可以與「Mortgage Assistant Agent」對話。

測試案例 1：「順利路徑」- 摘要資料並遮蓋 PII

這項測試會驗證代理程式是否能透過 Agent Gateway 存取後端系統，以及是否會強制執行資料遺失防護 (DLP) 政策，遮蓋機密資訊。

1. 向抵押貸款助理代理傳送下列提示：

I'm reviewing the Sterling family's current application. Can you summarize their 2024 and 2025 tax returns and verify if their total household income meets our 2026 debt-to-income requirements?

2. 幕後到底發生了什麼事？

- Gemini Enterprise 中的代理程式會向必要工具發出要求 (例如稅務申報表的 DMS、所得驗證 API)。
- **Model Armor** 會檢查要求和回應酬載，找出威脅。
- 您設定的「Content-based Authorization Policy」會觸發自訂資料遺失防護擴充功能 (`dlp-content-authz-ext`)。這項擴充功能會檢查從後端系統擷取的資料。
- DLP 服務會先從報稅資料中遮蓋任何個人識別資訊 (PII)，例如社會安全號碼 (SSN)，再將資料傳送給服務專員。

3. **預期結果：**代理程式會傳回退稅摘要和所得驗證狀態。請務必檢查代理程式提供的摘要。您應該會發現，納稅人 ID (SSN) 等機密資訊已替換為預留位置 (例如 `[REDACTED]`)。這表示閘道已執行 DLP 政策。

可觀測性和稽核

在這些測試期間，代理程式平台和相關服務會收集遙測資料：

- **Cloud Logging：**GKE Gateway、GKE 工作負載和其他服務的詳細記錄，可提供要求、政策評估和結果的稽核追蹤記錄。
- **Cloud Trace：**代理程式和後端服務中設定的 OpenTelemetry 檢測功能，可讓您將整個呼叫流程視覺化，從 Gemini Enterprise 透過 GKE Gateway 到後端工具。這對偵錯和瞭解要求生命週期至關重要。

查看工作階段的追蹤記錄：

1. 在 Google Cloud 控制台中，前往 Vertex AI Agent Engine 頁面。
2. 前往 [Agent Engine](#)。清單中會顯示所選專案的 Agent Engine 執行個體。您可以使用「篩選」欄位，依指定欄位篩選清單。

3. 按一下 Agent Engine 執行個體的名稱。
 4. 按一下「追蹤」分頁標籤。
 5. 您可以選取「工作階段」或「時間範圍」檢視畫面。
 6. 點選工作階段或時距，即可查看追蹤記錄詳細資料，包括時距的有向無環圖 (DAG)、輸入和輸出內容，以及中繼資料屬性。
-

10. 選用：使用 Apigee 將 REST API 轉碼為 MCP

雖然我們的收入驗證服務原生支援 Model Context Protocol，但許多企業舊版系統只提供 RESTful API。在這個選用步驟中，您將使用 **Apigee MCP Discovery Proxy**，將收入驗證服務的 REST 端點轉碼為 MCP 工具。這樣一來，您就能將 Apigee 的進階管理、頻率限制和安全性政策套用至舊版工具。

詳情請參閱 [Apigee MCP 總覽](#)。

必要條件

請先佈建及設定 **Apigee API 中心**，再繼續操作。按照「[佈建 API 中心](#)」說明文件中的步驟進行設定，並連結 Apigee 執行個體。

步驟 1：為 Apigee 建立服務附件

如要允許 Apigee 存取在 GKE 上執行的內部服務，您必須建立服務連結。

1. 查詢內部 GKE Gateway 轉送規則：

```
export RULE_URI=$(gcloud compute forwarding-rules list \
  --filter="loadBalancingScheme=INTERNAL_MANAGED AND target~targetHttpsProxies" \
  --format="value(selfLink.segment(6), region.basename(), name)" | \
  awk '{print "projects/" $1 "/regions/" $2 "/forwardingRules/" $3}')
```

2. 建立服務連結：

```
gcloud compute service-attachments create internal-gke-gateway-apigee \
  --region=${REGION} \
  --target-service=$RULE_URI \
  --connection-preference=ACCEPT_AUTOMATIC \
  --nat-subnets=gateway-psc-subnet
```

步驟 2：使用 Terraform 啟用 Apigee

現在，請更新基礎架構設定，佈建 Apigee 機構和執行個體。

1. 請前往 `terraform` 目錄：

```
cd terraform
```

2. 編輯 `terraform.tfvars` 並設定 `enable_apigee = true`。
3. 套用變更：

```
terraform apply
```

注意：初始 Apigee 組織佈建作業可能需要 30 到 50 分鐘。

步驟 3：定義 OpenAPI 規格

Apigee 會使用標準 OpenAPI (OAS) 定義來探索及轉碼工具。建立名為 `income-verification-oas.yaml` 的檔案，並加入以下內容：

```
openapi: 3.0.0
info:
  title: Income Verification API
  description: Verify applicant income through third-party employer records.
  version: 1.0.0
servers:
  - url: https://income-verification.internal.${DNS_PEERING_DOMAIN%%.}/api
paths:
  /income-verification/verify:
    post:
      summary: Verify applicant income
      operationId: verifyApplicant
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                first_name:
                  type: string
                last_name:
                  type: string
      responses:
        '200':
          description: Successful verification
          content:
            application/json:
              schema:
                type: object
```

步驟 4：部署 Apigee MCP Discovery Proxy

MCP 探索 Proxy 是專用的 Apigee Proxy，可做為 MCP 伺服器。

1. 在 Apigee UI 中，依序前往「開發」>「API Proxy」。
2. 按一下「建立新項目」，然後選取「MCP 探索 Proxy」。
3. 將 Proxy 命名為 `income-verification-discovery`。
4. 在「OpenAPI Spec」(OpenAPI 規格) 區段中，上傳您建立的 `income-verification-oas.yaml` 檔案。
5. 將「環境群組」設為可存取內部閘道的群組。
6. 按一下 [Deploy] (部署)。

(選用) 新增安全性政策

部署或共用 Proxy 前，請務必先保護 Proxy 安全。您可以新增政策來強制執行安全性規定，例如 OAuth 權杖或 API 金鑰。如需新增安全防護政策的操作說明，請參閱 [Apigee 說明文件中的 API 安全防護機制](#)。

步驟 5：確認轉碼工具存取權

部署完成後，Apigee 會自動將 `POST /verify` 端點轉碼為 `verifyApplicant` MCP 工具。

1. 列出透過 Apigee MCP 端點提供的工具：

```
curl -X POST https://api.internal.${DNS_PEERING_DOMAIN%}/income-verification-
discovery/mcp \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}'
```

2. 您應該會在回應中看到 `verifyApplicant` 工具，這是從 REST 規格轉碼而來。您現在可以透過 Apigee 呼叫這項工具，Apigee 會處理基礎 REST 服務的翻譯作業，同時套用您設定的所有安全性政策。

步驟 6：更新 Mortgage Agent 以使用 Apigee

Apigee 已成功將 REST API 轉碼為符合 MCP 規範的工具，現在您必須更新代理程式的部署設定。將代理程式指向 Apigee 端點後，所有收入驗證要求都會受到 Apigee 企業級安全防護、記錄和流量管理功能的保護。

1. 找出 **Apigee MCP 網址**：端點應符合 `https://api.internal.${DNS_PEERING_DOMAIN%}/income-verification-discovery/mcp` 模式。
2. 重新執行部署指令碼：使用 `--update` 旗標和新的 `--income-verification-url`。這會更新 Agent Engine 中的現有代理程式，不必完全刪除。

```
cd src/mortgage-agent

# Update the agent to route income verification through Apigee
uv run python deploy_agent.py \
  --project=${PROJECT_ID} \
  --update \
  --agent-name=mortgage-agent \
  --dms-mcp-url=https://dms.${DNS_PEERING_DOMAIN%}/mcp \
  --income-verification-url=https://api.internal.${DNS_PEERING_DOMAIN%}/income-
verification-discovery/mcp \
  --email-mcp-url=https://email.${DNS_PEERING_DOMAIN%}/mcp \
  --network-
attachment=projects/${PROJECT_ID}/regions/${REGION}/networkAttachments/${PSC_ATTACHMENT}
\
  --dns-peering-domain=${DNS_PEERING_DOMAIN} \
  --dns-peering-target-project=${PROJECT_ID} \
  --dns-peering-target-network=${VPC_NAME} \
  --ge-deploy \
  --app-id=${APP_ID} \
  --oauth-client-id=${OAUTH_CLIENT_ID}
```

使用 Apigee MCP Discovery Proxy 時，代理程式會將轉碼後的 REST API 視為原生 MCP 伺服器。 `deploy_agent.py` 指令碼會將這些工具註冊到 Agent Engine 環境。

3. 驗證變更：返回 Gemini Enterprise 介面，要求代理程式驗證申請人的收入。

```
"Can you verify the joint income for the Sterlings using the verification service?"
```

在 **Apigee Debug** 分頁中，您現在應該會看到來自 GKE Gateway 的 JSON-RPC 要求轉換為標準 REST `POST` 要求，傳送至後端 GKE 服務。

11. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所建立資源的費用，請在完成後刪除這些資源。

1. 如果服務附件是手動建立，請刪除：

```
gcloud compute service-attachments delete internal-gke-gateway \
  --region=${REGION} --quiet
```

2. 前往 `terraform` 目錄並刪除所有資源：

```
cd terraform
terraform destroy
```

3. (選用) 刪除整個 Google Cloud 專案：

```
gcloud projects delete ${PROJECT_ID}
```

12. 恭喜

您已完成本程式碼研究室，並瞭解如何保護跨雲端代理企業部署作業。

涵蓋範圍

- 已將 ADK 房貸助理代理部署至 Agent Engine，並使用 OpenTelemetry 檢測
- 將後端 MCP 伺服器部署至內部閘道後方的 GKE
- 使用 PSC 介面和 DNS 對等互連，將 Agent Engine 連接至專案 VPC
- 設定 GKE Gateway，以受控代理程式輸出內容，並使用 DLP 遮蓋和 MCP 授權
- 使用 [Model Armor](#) 應用 AI 防護機制，篩選提示詞和回覆
- (選用) 使用 Apigee MCP Discovery Proxy 將 REST API 轉碼為 MCP

後續步驟

- 探索 [Model Armor](#)
 - 瞭解 [Agent Engine](#)
 - 探索 [Gemini Enterprise](#)
-